

HILMA-Res: A General Hierarchical Framework via Residual RL for Combining Quadrupedal Locomotion and Manipulation

Xiaoyu Huang¹, Qiayuan Liao¹, Yiming Ni¹, Zhongyu Li¹, Laura Smith¹,
Sergey Levine¹, Xue Bin Peng², Koushil Sreenath¹

Abstract—This work presents HILMA-Res, a hierarchical framework leveraging reinforcement learning to tackle manipulation tasks while performing continuous locomotion using quadrupedal robots. Unlike most previous efforts that focus on solving a specific task, HILMA-Res is designed to be general for various loco-manipulation tasks that require quadrupedal robots to maintain sustained mobility. The novel design of this framework tackles the challenges of integrating continuous locomotion control and manipulation using legs. It develops an operational space locomotion controller that can track arbitrary robot end-effector (toe) trajectories while walking at different velocities. This controller is designed to be general to different downstream tasks, and therefore, can be utilized in high-level manipulation planning policy to address specific tasks. To demonstrate the versatility of this framework, we utilize HILMA-Res to tackle several challenging loco-manipulation tasks using a quadrupedal robot in the real world. These tasks span from leveraging state-based policy to vision-based policy, from training purely from the simulation data to learning from real-world data. In these tasks, HILMA-Res shows better performance than other methods.

I. INTRODUCTION

Using legs as manipulators to perform non-prehensile manipulation tasks is a natural behavior commonly observed among humans. Beyond just locomotion, people are capable of using their legs for a variety of actions, such as dribbling a soccer ball while walking or running, or moving an object by kicking it rather than lifting. Enabling legged robots, like quadrupeds, to perform such loco-manipulation tasks with their legs while moving could significantly enhance their versatility and applicability. However, this is a hard problem. Using the quadrupedal robot as an example, the robot needs to consider its stability while walking and also has to adjust its legs and use whole-body maneuvers to accomplish fine-grained manipulation tasks. The robot has to update its leg movement strategy not only for locomotion control but also for manipulation planning in real-time. Developing a general solution for different loco-manipulation tasks further complicates this due to the distinct objectives and environments associated with different manipulation tasks. Many attempts have been made to address the challenge of loco-manipulation, which involves the coupled problem of locomotion control and manipulation planning. Due to its difficulty, many efforts have opted to simplify the scenario. This simplification includes focusing on a single task, such as just ball dribbling [1], [2], or developing separate controllers



Fig. 1: The proposed HILMA-Res framework enables a quadrupedal robot to perform different loco-manipulation tasks in the real world. These include dribbling a ball in a desired direction, stepping over small blocks scattered on the ground, and navigating a load to the desired goal through real-world learning. We highlight the versatility of the HILMA-Res framework in various loco-manipulation tasks with different observation spaces and learning algorithms.

like [3]–[5]: one for manipulation using the robot’s legs and another for continuous locomotion skills. It is still an open question to develop a general framework that can solve different manipulation tasks using the robot’s legs while performing continuous locomotion. In this work, we propose a hierarchical framework, named HILMA-Res, to divide the complex loco-manipulation task and conquer the control and planning sub-problems individually by reinforcement learning (RL). It can be applied to different loco-manipulation tasks in the real world, as shown in Fig. 1.

A. Contributions

The main contribution of this work is the introduction of HILMA-Res. (1) It is a general framework enabling quadrupedal robots to perform various manipulation tasks while maintaining sustained mobility through hierarchical reinforcement learning (RL). (2) The novel design of HILMA-Res addresses the challenges of integrating manipulation skills with continuous locomotion. This includes implementing a single task-independent operational space locomotion controller, such as a walking controller, to track arbitrary end-effector (toe) trajectories, and a high-level task-specific manipulator planner for determining a residual trajectory for the end-effector. (3) We demonstrate that this design offers a versatile solution for different loco-manipulation tasks, requiring minimal effort for retraining on individual tasks. This framework supports different observation spaces and RL algorithms. (4) Our experiments showcase the ability of HILMA-Res by realizing various loco-manipulation tasks in the real world, including ball dribbling (training from simulation data), stepping over stones (using visuomotor skills), and load navigation (training from real-world data). Additionally, we show that HILMA-Res outperforms other state-of-the-art RL methods in the challenging loco-manipulation task.

¹ University of California, Berkeley, USA

² Simon Fraser University, Canada

Email: haytham.huang@berkeley.edu

II. RELATED WORK

Different from the efforts on mobile manipulation with wheeled bases which does not need to consider the robot’s stability [6], [7], loco-manipulation using legged robots requires different solutions. Previous work mainly focuses on two approaches: arm-equipped legged robots, and using the robot’s legs as manipulators.

Mobile Manipulation with Legged Robots: Equipping a legged robot, like a quadruped, with an additional arm can expand its manipulation capabilities by utilizing its mobility and whole-body maneuvers. Previous research has explored various methods to improve the coordination between locomotion and manipulation. Some have proposed a residual learning framework to enhance skill coordination [8], requiring a pre-existing locomotion controller. Others have focused on developing a unified whole-body policy for both manipulation and locomotion [9]–[11], though without incorporating high-level planning for task space, and focusing solely on joint-level control. Model-based approaches, using predictive and whole-body control, facilitate coordinated movements by accounting for multi-rigid body dynamics but depend on predefined contact sequences and task-specific constraints [12], [13]. Techniques combining trajectory optimization and contact planning have achieved versatile multi-contact loco-manipulation, albeit with the necessity for off-line planning [14]. However, integrating an arm increases the system’s complexity and necessitates additional hardware, more degrees of freedom, and the added load.

Using Legs for Both Locomotion and Manipulation: Attempts leveraging legs as end effectors for manipulation using legged robots could have simplicity in hardware but limit tasks to non-prehensile manipulation [15]. Legs are critical for stabilizing the robot, and using them for extra manipulation tasks poses challenges in maintaining stable gaits. Some work avoids the need for precise manipulation by only focusing on manipulating large objects [16]. To directly use legs for manipulation tasks, some prior work used two cascaded RL-based policies for control and planning in the robot’s operational space, respectively. This method enables a quadrupedal robot to shoot [4] or intercept [5]. Others opt to develop a unified policy. However, these simplify the loco-manipulation problem by the constrained mobility, such as just standing [4] or only jumping once [5]. In other work like [3], a single policy is developed for individual loco-manipulation tasks that are still limited to constrained mobility, such as standing against a wall to push a button. If there is a need for continuous locomotion, such as walking, to enlarge the robot’s working space, a separate locomotion controller and rule-based policy selector are necessary in this work [3]–[5]. Specialized tasks, like dribbling soccer balls while walking, have seen some development but lack scalability [2]. In the humanoid robotics field, besides heuristic-based approaches like [17], hierarchical learning [1] framework has been explored for using legs in manipulation tasks, yet this work focuses on task-specific training without a generalizable low-level controller for downstream

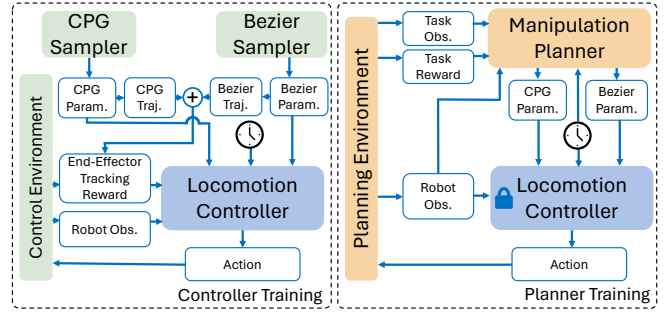


Fig. 2: The HILMA-Res framework. This hierarchical framework consists of a controller training stage where we train a task-independent locomotion controller that tracks desired end-effector trajectories. This is an addition from sampled CPG trajectories and sampled Bézier residual trajectories. A planner training stage is designed to reuse the locomotion controller to train a task-specific manipulation planner for downstream loco-manipulation tasks. We highlight the importance of reusing a pre-trained locomotion controller that has been evaluated in the real-world environment, which enables fast and efficient learning of the planner and prevents learning dynamically infeasible actions.

manipulation tasks. This work introduces HILMA-Res, a hierarchical framework aiming for generality across loco-manipulation tasks, addressing these limitations.

III. THE HILMA-RES FRAMEWORK AND LOCO-MANIPULATION TASKS

In this section, we provide an overview of the HILMA-Res framework, as visualized in Fig. 2. We frame the loco-manipulation tasks with continuous locomotion as a combination of two problems with different levels of difficulty of generalization using RL: versatile locomotion control that can be realized by a single policy and object manipulation that can be hard to generalize to different tasks.

The HILMA-Res contains two parts: a task-independent operational space locomotion controller and a task-specific manipulation planner, as shown in Fig. 2. The locomotion controller is responsible for joint-level control of the whole body of the quadrupedal robot, leveraging the robot’s proprioceptive feedback as input. We adopt a motion tracking method where a parameterized reference motion provided by a Central Pattern Generator (CPG) [18] is utilized to provide gait priors for tracking various commanded velocities. Note that the CPG is designed to generate end-effector (toe) trajectories for the quadrupedal robot with different contact sequences, different walking velocities, and turning rates. This facilitates planning and control in the operational space. Residual trajectories, represented by Bézier curves, are added to the nominal end-effector trajectories of the *swing legs* from CPG, while leaving the stance leg unchanged. The details for training this operational space locomotion controller are described in Sec. IV. After the locomotion controller is available and evaluated in the real world, it can be kept and re-used by different task-specific planners. The planner is designed to accomplish specified loco-manipulation tasks by specifying the residual trajectories of the end-effector and the commands for the robot moving base for the locomotion controller. The input of the planner includes not only the

robot’s proprioceptive feedback but also task-specific information, such as the object’s location, or directly using depth vision as input. Moreover, the planner can be constructed through different methods, as described in Sec. V.

To demonstrate the versatility of the HILMA-Res framework, we evaluate our method on three distinct loco-manipulation tasks, with increase in difficulty:

Ball Dribbling (Dribble): In this task, a quadrupedal robot dribbles a soccer ball and controls the ball’s velocity and direction while dribbling, using a vision-based ball detection with onboard cameras.

Stepping over Stones (StepOStone): This task requires a quadrupedal robot to walk through a path littered with small stones or ground obstacles without collisions, inspired by the agile movements of a cat (as seen in [19], and as tackled by a quadrupedal robot in simulation only in [20]). Since measuring the positions of the small stone could introduce significant errors, we directly leverage the robot’s onboard vision in the manipulation planner to prevent the robot’s end-effectors from hitting the stones. This could allow the policy developed in simulations to be applied in the real world.

Navigating Load (NavLoad): the robot uses its front legs to push a small yet relatively heavy box towards a specified target location, requiring strategic long-term planning. The difference between simulated environments and the real world (like ground friction, size and stiffness of the load) can lead to accumulated errors over the long horizon. Therefore, this task necessitates real-world training to successfully and efficiently push the load to the target.

These tasks showcase a wide range of loco-manipulation capabilities: (1) the use of visual observation (**StepOStone**) versus state observation (**Dribble**, **NavLoad**), (2) the necessity for short-term planning (**StepOStone**, **Dribble**) versus long-term planning (**NavLoad**), and (3) tasks that demand training with real-world data (**NavLoad**) as opposed to those that can be transferred directly from simulation to the real world (**StepOStone**, **Dribble**).

IV. TASK-INDEPENDENT QUADRUPEDAL LOCOMOTION CONTROL IN OPERATIONAL SPACE

The first component of HILMA-Res is an RL-trained quadrupedal locomotion controller designed to be broadly applicable to a wide range of downstream loco-manipulation tasks. The locomotion controller is trained to enable the robot to follow a *diverse* set of end-effector trajectories for the swing legs while walking, *i.e.*, operating in the operational space. *For brevity, we use “trajectories” to refer to “end-effector trajectories of the robot’s swing legs”.*

A. Parameterized Reference Trajectory in Operational Space

We first provide a parameterized reference trajectory. Such a parameterized reference trajectory is the addition of two parts: (1) nominal trajectories for a trotting gait from a Central Pattern Generator (CPG), and (2) residual trajectories represented by Bézier trajectories that are added to the nominal trajectories, as illustrated in Fig. 3. Given a trotting gait, there are two nominal and residual trajectories for

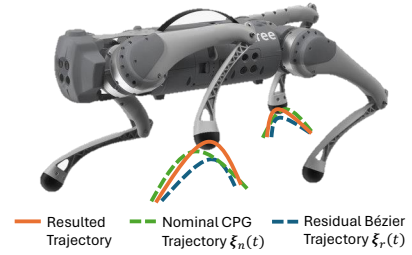


Fig. 3: In this work, the quadrupedal robot employs a trotting gait, characterized by two diagonal legs swinging simultaneously. For each swing leg, there is a desired end-effector trajectory, which is a summation of a nominal CPG trajectory ξ_n and a residual Bézier trajectory ξ_r . By learning to change the control points of Bézier trajectories and the base movement, the policy could adjust the swing trajectories to perform various loco-manipulation tasks.

the two swing legs, respectively. All the trajectories are defined in the base footprint frame of the robot. Having this design for the trajectories is the key to developing a versatile framework to enable the robot to learn to walk while adjusting the swing leg trajectories for manipulation tasks.

1) *Nominal Trajectories from CPG:* The Central Pattern Generator (CPG) is a bio-inspired way to obtain swing foot trajectories, which is commonly used in quadrupedal locomotion control. CPG represents the trajectories of all four feet in 3D Cartesian space by high-order polynomials that are parameterized by desired base velocity $\dot{q}_{x,y}^d$ and turning yaw rate $\dot{q}_\psi^d$, with a given step height at the apex for swing legs. By using a periodic oscillator, the contact sequence of each leg can be obtained by choosing different oscillator parameters, which results in different rhythmic gaits, such as trotting, pacing, and etc. For more details of CPG, we refer readers to [18, Sec. III]. For simplicity, we only use CPG to develop a trotting gait by fixing the parameters of the oscillator in this work. In this way, with a fixed gait period T_g , by varying the given command, the periodic trajectories $\xi_n(t)$ can be well defined with respect to the time t . These serve as nominal trajectories for the operational space locomotion controller. We further denote $\dot{q}_{x,y,\psi}^d$ as CPG parameters which could be varied in this work.

2) *Residual Trajectories Represented by Bézier Curves:* Bézier curves are utilized to specify the change of the nominal trajectories from CPG, representing the residual trajectories $\xi_r(t)$. In this work, the Bézier curves are parameterized by three control points $\mathbf{P}_i$ ($i = 0, 1, 2$) in 3D Cartesian space. To ensure the swing legs’ trajectories start and end on the ground (with vertical displacement $z = 0$), we set the vertical placement of the first and last control points to zero, leveraging the characteristic of Bézier curves to always pass through these points. In this way, the Bézier parameters that can be chosen are $\mathbf{P}_{0,2} \in \mathbb{R}^2$ and $\mathbf{P}_1 \in \mathbb{R}^3$. Then the change of position in the trajectory at time t can be well defined by the Bézier function $\xi_r(t)$. The time t needs to be first normalized to be within $[0, 1]$ w.r.t. the timespan of the Bézier trajectory (*i.e.*, the robot swing period $T_{sw} = T_g/2$).

3) *Control Objective:* The objective of this locomotion controller at each time t is to track the desired end-effector trajectories of the swinging legs, $\xi_n(t) + \xi_r(t)$, while tracking

the commands for robot base, such as walking velocity and turning yaw rate (the CPG parameters $\dot{q}_{x,y,\psi}^d$).

B. Control Environment

The control policy is trained via RL in a simulated environment to track target end-effector trajectories and robot base commands while trotting.

1) *Action*: The control policy’s output actions specify desired motor positions $\mathbf{a}_k^c$ at each timestep k . The policy is queried at 50 Hz, and the target motor positions are then utilized in a joint-level PD controller running at 1 kHz to calculate motor torques.

2) *Observation*: At each timestep k , the control policy receives an observation from the environment and determines an appropriate action for that scenario. The observation $\mathbf{o}_k^c$ is the robot’s proprioceptive feedback, which includes the robot’s current angular position $q_{\phi,\theta,\psi}$, angular velocity $\dot{q}_{\phi,\theta,\psi}$, motor position $\mathbf{q}_m$, and motor velocity $\dot{\mathbf{q}}_m$. Alongside the current observation, we also keep track of a short 4-timestep history of the robot’s past input (action) and output (observation), *i.e.*, $(\mathbf{o}_{k:k-4}^c, \mathbf{a}_{k-1:k-4}^c)$, which are provided as input to the control policy. This short I/O history helps to facilitate state estimation and system identification. Additionally, we include a phase variable $\varphi = 2\pi t/T_g$ for the control policy to indicate the time t of one period of a trotting gait T_g . The phase variable is encoded using a smooth sinusoidal representation ($\sin \varphi, \cos \varphi$).

3) *Goal*: In addition to the observations, the policy is also conditioned on a goal, which consists of two components. The first component is the desired end-effector trajectories to track, which are altered by Bézier parameters $\mathbf{P}_{0,1,2}$ for each swing leg. The second part includes the command for the robot’s moving base, including desired sagittal velocity $\dot{q}_x^d$, lateral velocity $\dot{q}_y^d$, and turning yaw rate $\dot{q}_\psi^d$, which are also parameters for the CPG.

4) *Reward*: At each timestep k , the robot executes an action $\mathbf{a}_k$ from the control policy. The environment then transitions to a new state and produces a reward r_k^c . The reward is a weighted composition of different terms. The most dominating term (with the largest weight) is the end-effector tracking reward, which encourages the robot to track the desired trajectories of the four feet. For swing legs, the reference trajectories are obtained in Sec. IV-A. For stance legs, the reference trajectories become a single point to keep the foot unmoved. This reward is formulated as $r_{\text{tracking}}^c = \exp(-\sigma_e \|\mathbf{x}_{e,t} - (\boldsymbol{\xi}_n(t) + \boldsymbol{\xi}_r(t))\|_2)$ where $\mathbf{x}_{e,t}$ is the robot end-effector position $\mathbf{x}_e$ at time t and $\sigma_e > 0$ is a hyperparameter. Furthermore, penalties are also introduced for roll, pitch angular velocities and vertical linear velocity to stabilize the robot base and yaw position tracking.

5) *Task Randomization*: To diversify training, we randomize the controller policy’s tasks by varying the goal, by sampling Bézier parameters and CPG parameters uniformly from ranges in Table I. Diverse trajectories are generated by altering these parameters every 10 seconds. Training policies to perform a diverse range of tasks can also improve the robustness of the learned models [21].

Bézier Parameters Randomization Range			
Control points	x Range [m]	y Range [m]	z Range [m]
$\mathbf{P}_0$	[-0.07, 0.03]	[-0.1, 0.1]	-
$\mathbf{P}_1$	[-0.035, 0.035]	[-0.1, 0.1]	[-0.05, 0.05]
$\mathbf{P}_2$	[-0.03, 0.07]	[-0.1, 0.1]	-
CPG Parameters Randomization Range			
Command	Range	Unit	-
$\dot{q}_x^d$	[-1., 1.]	m/s	-
$\dot{q}_y^d$	[-0.3, 0.3]	m/s	-
$\dot{q}_\psi^d$	[-1., 1.]	rad/s	-

TABLE I: Range of Bézier parameters (control points) and CPG parameters used during training.

6) *Dynamics Randomization*: Since the training environment is built in simulation, we also include dynamics randomization [22] to facilitate transfer from simulation to the real world. The randomized dynamics parameters include joint PD gains, ground friction, base mass, and random perturbations, whose ranges are detailed in [23, Table 3].

C. Training and Deployment Details

The episode is designed to last 4000 timesteps or 80 seconds, trained with GPU-accelerated Isaac Gym physics simulator. The policy uses a three-layer MLP with hidden sizes of [128, 64, 32] and ELU activation, optimized by PPO [24]. After convergence, the obtained locomotion controller can be zero-shot transferred to the robot hardware.

V. TASK-SPECIFIC QUADRUPEDAL MANIPULATION PLANNING FOR RESIDUAL TRAJECTORIES

After obtaining the operational space locomotion controller that tracks arbitrary end-effector trajectories on the quadruped hardware, we can reuse this controller for different downstream manipulation tasks. Given the diverse nature of the manipulation tasks, as described in Sec. III, we choose to develop task-specific high-level planning policies to specify the goal for the task-independent control policy.

A. Planning Environment

In this work, we use RL to develop the planning policy. As a note, HILMA-Res allows us to leverage alternative approaches, like learning from demonstrations, for solving manipulation planning.

1) *Task-independent Action*: The manipulation planner determines the end-effector trajectories and robot base movement for the controller based on the given task. Specifically, the planner action $\mathbf{a}_k^p$ at each timestep includes Bézier parameters for swing legs’ trajectories (control points $\mathbf{P}_{0,1,2}$) and CPG parameters (desired base velocity and turning rate $\dot{q}_{x,y,\psi}^d$). The selection of Bézier parameters (for residual trajectories) solely influences end-effector trajectories, while CPG parameters (for nominal trajectories) alter desired base movement. This action space ensures compatibility with a task-independent locomotion controller across various tasks.

2) *Task-independent Observation*: The planner’s observation consists of task-independent elements consistent across tasks, and task-specific elements unique to individual tasks. Task-independent observations include the robot base’s measurable feedback, including its angular position and velocity, and the phase pair $(\sin \varphi, \cos \varphi)$ for synchronizing with the low-level locomotion controller’s periodic gait.

3) *Additional Task-specific Observation*: Each task-specific planner requires unique observations from the environment, either state or sensory feedback. HILMA-Res allows us to flexibly manage both observation types.

a) *State Representation*: In *Dribble* task, the robot uses the relative position of the ball and the global heading of the robot to dribble the ball, conditioned on the given desired ball velocity (both speed and angle). For *Dribble*, where controlling the ball’s velocity is crucial, we input a 4-timestep history of the robot and ball positions into the policy to infer the ball velocity without relying on noisy velocity estimates. In *NavLoad* task, the robot is provided with robot pose and load position, conditioned on the navigation target, all in the global frame, to navigate the load to the target. We do not provide history as input in this position-based task.

b) *Vision Representation*: In the task of *StepOStone*, obtaining an accurate state estimator and acquiring a precise height map that correctly labels the scattered stones could be difficult on a quadrupedal robot with onboard sensors. Therefore, the robot directly leverages its raw and downsampled depth vision from the ego view to learn to extract features related to the task.

4) *Reward*: The planning agent’s reward r_k^p at each timestep k is task-specific within the hierarchical framework. Thanks to the HILMA-Res, we are able to provide *simple* task objectives without worrying the complexity of motion control, streamlining task learning and minimizing complex reward-tuning efforts present in end-to-end methods.

a) *Dribble*: The reward is to encourage minimizing the tracking error of the desired linear velocity and moving direction of the ball. We also include a proximity term to encourage the robot to stay close to the ball. Specifically, $r_{\text{close}}^p = \exp(-\sigma_{\text{ball}} d_{\text{ball}})$ where d_{ball} is the distance between the robot and the ball and $\sigma_{\text{ball}} > 0$ is a hyperparameter.

b) *StepOStone*: The rewards are given for not stepping on stones or contacting with its calf. Specifically, $r_{\text{contact}}^p = 0$ if any leg is in contact, 1 otherwise. Similarly, $r_{\text{stepping}}^p = 0$ if any leg steps on an object, 1 otherwise. The weights are 0.7, 0.3 respectively.

c) *NavLoad*: We encourage the change of the distance between the load and the goal location. $r_{\text{move_load}}^p = \Delta d_{\text{load}} = d_{k-1, \text{load}} - d_{k, \text{load}}$, where d is the Euclidian distance between the goal and the load. We also include a proximity term formulated the same as the dribbling task, i.e., $r_{\text{close}}^p = \exp(-\sigma_{\text{load}} d_{\text{load}})$.

B. Training for Different Tasks

Using the HILMA-Res, we explore different setups of the RL training, ranging from more capable but data-hungry

algorithms that are mainly limited to simulation to data-efficient methods that can leverage real-world data.

1) *Tasks for State-based Policy*: Since the *Dribble* planner incorporates only a short history of the ball’s position, we use an MLP with three hidden layers of dimensions [128, 64, 32] and ELU activation. Trained via PPO in Isaac Gym, we simulate a dragging force on the ball introduced in [2] to facilitate zero-shot transfer from simulation to the real world.

2) *Tasks for Vision-based Policy*: Since the *StepOStone* policy directly uses depth vision, we firstly process the visual input via a CNN encoder with the first layer being convolution of size ([kernel, filter]) [5, 32], second layer being Max Pooling of size ([kernel, stride]) [2, 2], third layer being convolution of [3, 64], ReLU activation, and a linear layer that maps the visual data into a latent space of hidden size of 64. Temporal features are later captured by a GRU with the same hidden size. It is then combined with state-based observation, and further processed by two MLP layers with hidden sizes of [64, 32] and ELU activation. Unlike prior works that have to first train in state space and then transition to vision via teacher-student methods due to their low sample efficiency [20], [25], our efficient controller allows for direct end-to-end training with depth input in simulation by PPO in just five hours wall time. After training, the vision-based planner can be directly transferred to the real world.

3) *Tasks that Requires Real-World Data*: In the *NavLoad* task, we use a data-efficient DroQ [26] algorithm to learn from real-world data. The actor network is an MLP whose first hidden layer has 128 ReLU units and the second has 64 Tanh units. The critic network is a larger MLP with [256, 128] hidden neurons, ReLU activation, and LayerNorm before the activation function. we leverage RLPD [27], [28] to fasten the real-world training. First, we train the policy in simulation with DroQ (denoted as the base policy) and use this base policy to perform rollouts in the real world. We collect a replay buffer of 10,000 transitions. Then, we train a new policy from scratch using DroQ whose replay buffer is combined with both data collected by the base policy and the newly collected data from the real world, with a ratio of 70% : 30%. This could facilitate efficient learning in the real-world environment as described in [27].

Till now, details of the development of the HILMA-Res have been introduced.

VI. EXPERIMENT RESULTS

In this section, we evaluate the HILMA-Res policies for different loco-manipulation tasks extensively in both simulation and the real world. Due to the space limitation, we use *NavLoad* task as an example for extensive benchmark, as it is more challenging, and use *Dribble* and *StepOStone* as extension examples in the real world.

A. Navigating a Load

We first benchmark the performance of different methods for the *NavLoad* task in high-fidelity Gazebo simulation which provides a controlled environment. Then, we report real-world training results, followed by a discussion on the benefits of HILMA-Res compared to the baseline methods.

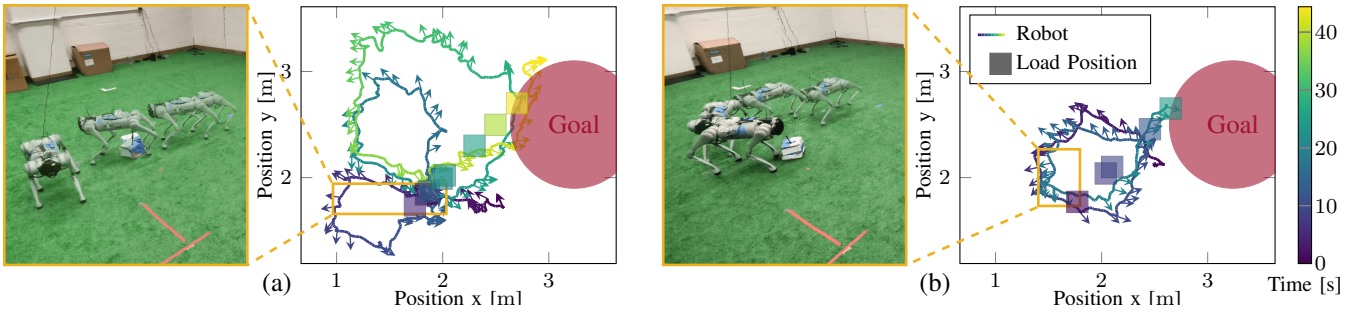


Fig. 4: Recorded data during *NavLoad* experiments, along with snapshots capturing the robot’s behavior during its first interaction with the load. (a) Zero-shot transfer of the base policy trained in simulation, the robot tends to have a large detour to move the load to the target due to a large sim-to-real gap (such as friction, sensor noise, etc). (b) Training with real-world data by RLPD, the robot first adjusts its pose and then pushes the load along the direction of the goal, with shorter path and operation time. This showcases the advantages of HILMA-Res in the fine-grained manipulation task that requires efficient training from real-world data.

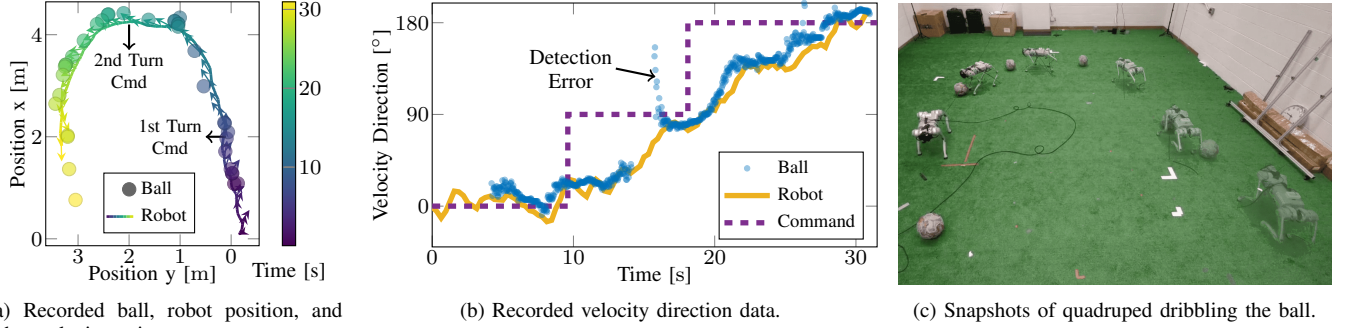


Fig. 5: Visualization of real-world ball dribbling experiments, using the proposed HILMA-Res framework. The quadruped can perform a sharp U-Turn within a narrow space of less than 3.3 meters in width. This demonstrates that the HILMA-Res can enable agile loco-manipulation maneuvers that can be directly transferred from simulation to the real world.

1) *Baseline*: We compare the HILMA-Res method against three popular *end-to-end* baselines on the *NavLoad* task. The baselines include:

- *Reward Shaping*: This baseline adopts a reward scheme from [29] without reference motions and adds task reward directly to the original reward terms.
- *Adversarial Motion Prior (AMP)*: This baseline is adopted from [30] and leverages adversarial imitation learning to learn from reference motions.
- *Motion Tracking (MT)*: This baseline learns locomotion by tracking a given end-effector trajectory while improving the task reward [21]. In implementation, we adopt the locomotion controller of HILMA-Res without Bezier residuals and CPG commands, and add task-specific observation and reward for this specific task.

These baselines reflect the state-of-the-art end-to-end locomotion controller, which has the potential to be extended to individual loco-manipulation tasks using task-specific policy. *We train each policy until it converges and achieves comparable success rates in training environments (Isaac Gym).*

2) *Simulation Benchmarking*: First, we benchmark the performance of HILMA-Res against other methods in a high-fidelity *Gazebo* simulation. Each method is tested for four different goals (1-meter in front, behind, left, or right of the load) with two attempts per goal, while the robot starts 1-meter behind the load at each reset. As shown in Table II, HILMA-Res outperformed all end-to-end comparison

methods in success rates. Specifically, the *Reward Shaping* method struggles to perform effective movement patterns during the sim-to-sim transfer, quickly losing balance and only kicking the load by chance. *AMP* manages to move when the load is distant but faces difficulties in stabilizing the body (being confused about which motion the robot should exert) as it approached the load, likely due to mode collapsing in adversarial learning. The *MT* baseline can move the loads to some extent but has trouble balancing motion imitation and task rewards, leading to unstable learning and evaluation performance. In contrast, HILMA-Res, with its stable locomotion controller and focused task planner, completes the task without issues, highlighting the advantage of a hierarchical approach for complex tasks.

3) *Real World Training and Evaluation*: We further test HILMA-Res for *NavLoad* in the real world. In the experiments, we utilize a localization system using Ultra-wideband (UWB) to measure the robot and load’s positions and the robot’s heading in the global framework. When we apply the simulation-trained base policy to real hardware, a notable drop in both success rates and efficiency is witnessed, due to the load’s shape not being a perfect cube as in the simulation and the high noise in localization when the robot gets close to the load, leading to inaccurate pushes and frequent misses, as shown in Fig. 4(a). To address these challenges, we utilize the RLPD algorithm to train with real-world data. The training from scratch with the RLPD algorithm

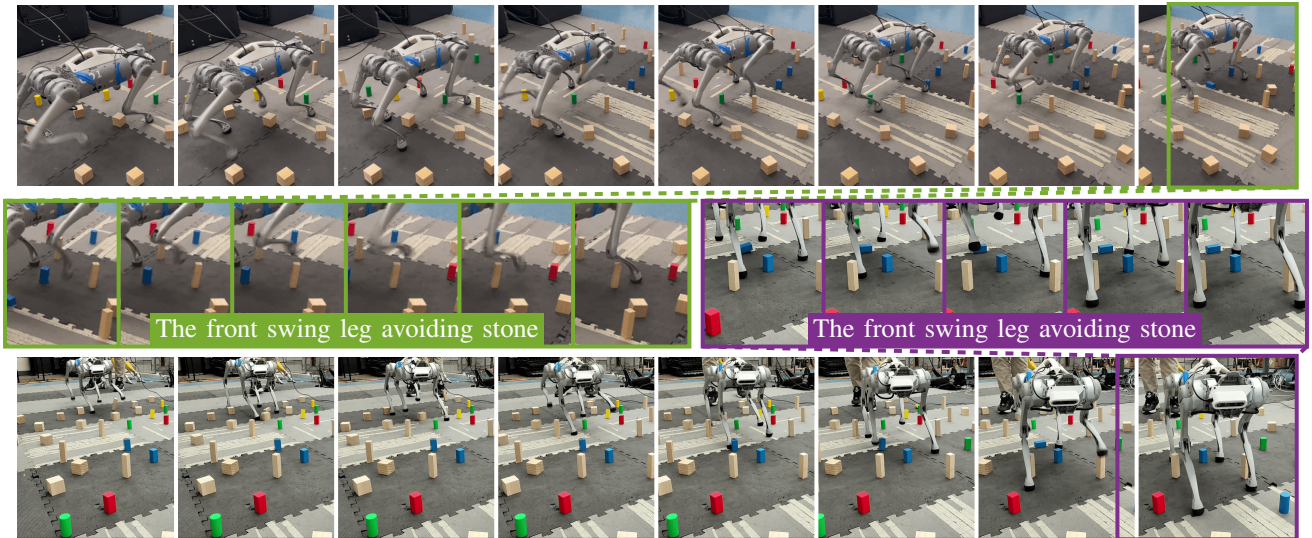


Fig. 6: Snapshots of two real-world experiments on *StepOStone*. The robot can walk through the randomly positioned small cluttered stones, employing a dynamic trotting gait. The zoom-in images emphasize the robot’s ability to sidestep obstacles by adjusting its swing legs. Note that some of the fall-down stones are caused by the impact variation when the legs make contact with the ground.

Method	Success Rate (%)	Average Time (s)
Sim-to-Sim Transfer		
Reward Shaping	25	1.9
AMP	0	/
Motion Tracking	62.5	40.94
Ours	100	23.2
Real World Experiments		
Ours (Base Policy)	80	57.75
Ours (Trained in Real)	100	47.6

TABLE II: Benchmark of the *NavLoad* task in sim-to-sim transfer and real experiments. We find that all end-to-end methods cannot achieve a high success rate in simulation other than the one it was trained on, highlighting the difficulty of achieving both robustness and high task performance in end-to-end training. In comparison, our method achieves a 100% success rate in simulation and can be transferred at an 80% success rate zero-shot in real environments. Furthermore, we are able to achieve a 100% success rate in the real environment after training with real data from scratch directly with RLPD algorithm.

takes just 9 minutes and 1,000 steps, leading to notable improvements in success rates and efficiency for real-world load navigation tasks. Compared to the base policy, which frequently misses the load and makes wide and inefficient turns, the policy trained with real-world data navigates more carefully. Shown in Fig. 4(b), this strategy increases effective kicks and successful load interactions. On missing the load, it also adjusts with tighter turns for faster retries, showing better robustness to sensor noise. These result in a shorter path to reach the target. This efficiency underscores the benefits of a hierarchical approach, enabling efficient and reliable real-world task learning.

B. Ball Dribbling

In this *Dribble* task, we use the robot onboard stereo fisheye cameras to detect the ball and calculate its rela-

Trial #	Stones Passed	Stones Fell	% Fell
1	22	2	9.09
2	20	3	15
3	19	2	10.5
4	19	3	15.8
Average	20	2.5	12.5

TABLE III: Results of four trials on the *StepOStone* task in real-world experiments. We report the number of stones the robot passed, and the number of stones which the robot steps on or hits. The robot avoids on average 87.5% of the stones.

tive position. With a proprioceptive state estimator, we can determine the robot’s heading in the world frame for this short-term task. Our results as shown in Fig. 5 demonstrate the dribbling planner’s effectiveness, capable of executing a sharp U-Turn with the ball in spaces as narrow as 3.3 meters. The heading profile indicates a smooth approach to the target direction. Despite a significant ball detection error at 16 seconds, the planner is still robust and goes directly to the ball’s position, with the help of its history observations that help to filter noisy measurements. The robot can dribble the ball at varying speeds, though the error in velocity tracking is large due to the gap in the physical properties of simulated and real soccer balls. Without a more accurate measurement system, we cannot learn the task directly in the real world due to noisy reward functions.

C. Step Over Stone

In the *StepOStone* task, we utilize a depth camera, down-sampled to 43×29 to match our training setup, and apply a simulation-trained policy directly in real-world tests. These tests involve randomly placed wood blocks ranging from 3cm to 5cm wide and 5cm to 10cm tall on a padded surface. In Fig. 6, on the center left snippets, the robot exhibits deliberate toe trajectories to bypass a tall wood

block by adjusting the CPG trajectories (nominally 10cm high, risking collision) using Bézier residuals, enabling it to step over and land safely beyond the obstacle. On the center right snippets, the planner adeptly retracts the left front leg, preventing collision with another tall block, unlike the expected path of the nominal curve. Quantitatively, the robot can 87.5% of the wood blocks on average across four trials, as recorded in Table III. These results demonstrate the proposed framework’s capability in mastering precise foot placements, notably in the *StepOStone* task, and underscore the locomotion controller’s accurate end-effector tracking.

VII. CONCLUSION AND FUTURE WORK

In conclusion, we present HIMLA-Res, a general hierarchical RL framework to tackle the loco-manipulation tasks. It combines a task-independent operational space locomotion controller that enables tracking control for both robot moving base and end-effector trajectories, and task-specific manipulation planners for downstream tasks. The HIMLA-Res is general and solves different loco-manipulation tasks. Through these tasks, we show the flexibility of the HIMLA-Res framework in dealing with real-world training, agile loco-manipulation, and precise foot placements with the help of the policy hierarchy and the Bézier residual learning. Compared to end-to-end methods, HIMLA-Res refrains the planner from overcoming sim-to-real challenges and benefits it with significantly higher sample efficiency.

Not limited to the presented tasks in this work, HILMA-Res has the potential for other tasks, such as button pushing. We also note that the framework can also be extended to different quadrupedal gaits other than trotting by training with more CPG parameters such as different contact patterns or stepping heights. In the future, we think HILMA-Res could also be useful for solving loco-manipulation tasks using humanoid robots.

REFERENCES

- [1] S. Bohez, S. Tunyasuvunakool, P. Brakel, F. Sadeghi, L. Hasenclever, Y. Tassa, E. Parisotto, J. Humpik, T. Haarnoja, R. Hafner *et al.*, “Imitate and repurpose: Learning reusable robot movement skills from human and animal behaviors,” *arXiv preprint arXiv:2203.17138*, 2022.
- [2] Y. Ji, G. B. Margolis, and P. Agrawal, “Dribblebot: Dynamic legged manipulation in the wild,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 5155–5162.
- [3] X. Cheng, A. Kumar, and D. Pathak, “Legs as manipulator: Pushing quadrupedal agility beyond locomotion,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [4] Y. Ji, Z. Li, Y. Sun, X. B. Peng, S. Levine, G. Berseth, and K. Sreenath, “Hierarchical reinforcement learning for precise soccer shooting skills using a quadrupedal robot,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.
- [5] X. Huang, Z. Li, Y. Xiang, Y. Ni, Y. Chi, Y. Li, L. Yang, X. B. Peng, and K. Sreenath, “Creating a dynamic quadrupedal robotic goalkeeper with reinforcement learning,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [6] J. Bohren, R. B. Rusu, E. G. Jones, E. Marder-Eppstein, C. Pantofaru, M. Wise, L. Mösenlechner, W. Meeussen, and S. Holzer, “Towards autonomous robotic butlers: Lessons learned with the pr2,” in *2011 IEEE International Conference on Robotics and Automation*, 2011.
- [7] Z. Fu, T. Z. Zhao, and C. Finn, “Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation,” *arXiv preprint arXiv:2401.02117*, 2024.
- [8] N. Yokoyama, A. Clegg, J. Truong, E. Undersander, T.-Y. Yang, S. Arnaud, S. Ha, D. Batra, and A. Rai, “Asc: Adaptive skill coordination for robotic mobile manipulation,” *IEEE Robotics and Automation Letters*, vol. 9, no. 1, pp. 779–786, 2023.
- [9] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, and X. Wang, “Expressive whole-body control for humanoid robots,” *arXiv preprint arXiv:2402.16796*, 2024.
- [10] Z. Fu, X. Cheng, and D. Pathak, “Deep whole-body control: Learning a unified policy for manipulation and locomotion,” in *Conference on Robot Learning*. PMLR, 2023, pp. 138–149.
- [11] Y. Wu, P. Balatti, M. Lorenzini, F. Zhao, W. Kim, and A. Ajoudani, “A teleoperation interface for loco-manipulation control of mobile collaborative robotic assistant,” *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3593–3600, 2019.
- [12] J.-P. Sleiman, F. Farshidian, M. V. Minniti, and M. Hutter, “A unified mpc framework for whole-body dynamic locomotion and manipulation,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, 2021.
- [13] H. Ferrolho, V. Ivan, W. Merkt, I. Havoutis, and S. Vijayakumar, “Roloma: Robust loco-manipulation for quadruped robots with arms,” *Autonomous Robots*, vol. 47, no. 8, pp. 1463–1481, 2023.
- [14] J.-P. Sleiman, F. Farshidian, and M. Hutter, “Versatile multicontact planning and control for legged loco-manipulation,” *Science Robotics*, vol. 8, no. 81, p. eadg5014, 2023.
- [15] Y. Gong, G. Sun, A. Nair, A. Bidwai, R. CS, J. Grezmak, G. Sartoretto, and K. A. Daltorio, “Legged robots for object manipulation: A review,” *Frontiers in Mechanical Engineering*, vol. 9, p. 1142421, 2023.
- [16] A. Rigo, Y. Chen, S. K. Gupta, and Q. Nguyen, “Contact optimization for non-prehensile loco-manipulation via hierarchical model predictive control,” in *International Conference on Robotics and Automation (ICRA)*, 2023.
- [17] H. Teixeira, T. Silva, M. Abreu, and L. P. Reis, “Humanoid robot kick in motion ability for playing robotic soccer,” in *International conference on autonomous robot systems and competitions*, 2020.
- [18] Y. Shao, Y. Jin, X. Liu, W. He, H. Wang, and W. Yang, “Learning free gait transition for quadruped robots via phase-guided controller,” *IEEE Robotics and Automation Letters*, 2021.
- [19] CatPusic, “Obstacle challenge for the cat. 3 levels,” <https://www.youtube.com/watch?v=gBwHSSS1pIA>, 2020.
- [20] S. Kareer, N. Yokoyama, D. Batra, S. Ha, and J. Truong, “Vinl: Visual navigation and locomotion over obstacles,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023.
- [21] Z. Li, X. B. Peng, P. Abbeel, S. Levine, G. Berseth, and K. Sreenath, “Reinforcement learning for versatile, dynamic, and robust bipedal locomotion control,” *arXiv preprint arXiv:2401.16889*, 2024.
- [22] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *International conference on robotics and automation (ICRA)*, 2018.
- [23] G. Feng, H. Zhang, Z. Li, X. B. Peng, B. Basireddy, L. Yue, Z. Song, L. Yang, Y. Liu, K. Sreenath *et al.*, “Genloco: Generalized locomotion controllers for quadrupedal robots,” in *Conference on Robot Learning*. PMLR, 2023, pp. 1893–1903.
- [24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [25] A. Agarwal, A. Kumar, J. Malik, and D. Pathak, “Legged locomotion in challenging terrains using egocentric vision,” in *Conference on robot learning*. PMLR, 2023, pp. 403–415.
- [26] T. Hiraoka, T. Imagawa, T. Hashimoto, T. Onishi, and Y. Tsuruoka, “Dropout q-functions for doubly efficient reinforcement learning,” *arXiv preprint arXiv:2110.02034*, 2021.
- [27] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient online reinforcement learning with offline data,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 1577–1594.
- [28] L. Smith, J. C. Kew, T. Li, L. Luu, X. B. Peng, S. Ha, J. Tan, and S. Levine, “Learning and adapting agile locomotion skills by transferring experience,” *arXiv preprint arXiv:2304.09834*, 2023.
- [29] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *Conference on Robot Learning*. PMLR, 2022, pp. 91–100.
- [30] A. Escontrela, X. B. Peng, W. Yu, T. Zhang, A. Iscen, K. Goldberg, and P. Abbeel, “Adversarial motion priors make good substitutes for complex reward functions,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.